

Enterprise Implementation Plan

SSO · Admin Dashboard · Invitations · White-Label

Single-instance per customer deployment

6 PHASES
Implementation Roadmap

Current State Baseline

✓ Working	■ Partial / Broken	✗ Not Built
28+ React pages JWT login (email/password) Sessions CRUD Recording + browser STT	AdminSettings UI (role gate broken) Workspaces + invites UI Usage tracking UI	User roles in DB/API Org admin dashboard Google / Apple SSO White-label runtime Invite emails

Target Role Model

Role	Who	Can Do
member	Normal org user	Record, sessions, personal usage, workspaces (if enabled)
org_admin	Customer IT / dept admin	Invite/disable users, assign roles, view all usage, export usage summary
system_admin	Deployment owner / super-admin	Everything org_admin + white-label settings, AI/platform settings, global limits

Note: First seeded user should be system_admin. Customer can promote others to org_admin.

Implementation Phases

**Backend —
Extend User
schema**

- role: member | org_admin | system_admin (default: member)
- status: invited | active | disabled (default: active)
- google_id, apple_id — for SSO linking
- invite_token_hash, invite_expires_at

New Models

- DeploymentSettings (singleton): app_name, logo_url, favicon_url, primary_color, accent_color, support_email
- Invite: email, role, token_hash, expires_at, invited_by, accepted_at

**Auth API
updates**

- GET /auth/me → include role, status, full_name
- POST /auth/register — disabled for public; only via invite / admin create
- Middleware: requireRole(...roles) in backend/src/middleware/auth.js

Frontend

- Fix admin gate — once /auth/me returns role, AdminSettings.jsx gate works
- Register AISettings in entityModels (or replace with DeploymentSettings)

Seed script

- First user → system_admin, status: active

Key files

backend/src/models/index.js backend/src/middleware/auth.js backend/src/routes/index.js
backend/scripts/seed-minimal.js

**New pages
under
/admin/org**

- Users — table: email, name, role, status, minutes/month, last active; actions: invite, change role, disable
- Usage — aggregate from PlanSubscription / session billable_minutes ledger
- Invites (tab) — pending invites, resend, revoke

**New APIs
(dedicated
routes)**

- POST /admin/users/invite — { email, role } → create invited user + token
- GET /admin/users — list with usage stats
- PATCH /admin/users/:id — role, status
- GET /admin/usage/summary — per-user and org total

Frontend wiring

- Wire Usage.jsx "coming soon" block → link org admins to /admin/org
- New AdminLayout.jsx (no consumer tab bar)
- requireRole guard on all /admin/org/* routes

Key files

src/pages/admin/OrgUsers.jsx src/pages/admin/OrgUsage.jsx
src/pages/admin/AdminLayout.jsx backend/src/routes/admin.js

**Invite flow
(sequence)**

- 1. Admin → POST /admin/users/invite
- 2. Backend → send invite email via emailService
- 3. User opens /accept-invite?token=... (7-day expiry, single-use)
- 4. User sets password OR SSO with matching email
- 5. Backend issues JWT → redirect /home

**Email system —
backend-owned
(not a SaaS
product UI)**

- emailService.js — sendTemplate(name, { to, vars })
- templateRegistry.js — register templates + validate vars
- layouts/baseLayout.js — shared HTML shell (table-based, inline CSS, 600px max)
- templates/userInvite.js — invite body HTML + plain text
- providers/resend.js — Resend adapter (recommended) | providers/ses.js | providers/console.js (dev)
- EmailLog model — audit: to, template, status, provider_id, sent_at, error

**Email provider
options**

- Resend (recommended v1): REST API, good deliverability, free tier for dev
- AWS SES: for gov/customers already on AWS
- console (dev only): EMAIL_PROVIDER=console → prints link to terminal

**Env vars
required
(Render)**

- EMAIL_PROVIDER=resend | ses | console
- RESEND_API_KEY=re_xxxx (if Resend)
- EMAIL_FROM=noreply@your-domain.gov
- EMAIL_FROM_NAME=Your App Name
- FRONTEND_URL=https://app.your-domain.gov

**HTML template
rules**

- Inline CSS only (Gmail strips external stylesheets)
- Max width ~600px, single primary CTA button
- alt on logo, mailto: for support link in footer
- Escape user-provided strings (inviterName, email) to prevent HTML injection
- Each template must export renderText(vars) — never HTML-only

Key files

backend/src/services/email/emailService.js
backend/src/services/email/templateRegistry.js
backend/src/services/email/layouts/baseLayout.js
backend/src/services/email/templates/userInvite.js
backend/src/services/email/providers/resend.js backend/src/models/EmailLog.js
src/pages/AcceptInvite.jsx

**Phase
04****Google & Apple SSO****MEDIUM****Backend routes**

- GET /auth/google → redirect to Google OAuth
- GET /auth/google/callback → find/create user by email; link google_id; issue JWT
- Same pattern for Apple (apple_id, Apple verifier)

Rules

- Invite-first policy (gov recommended): SSO only works if email has invited/active user — no open self-registration
- Linking: If user exists with password, first Google login with same email links google_id (optional confirm-link step)

**Frontend —
Login.jsx**

- "Continue with Google" / "Continue with Apple" buttons below email form
- Each customer registers OAuth apps with their own domain

**Env vars
required
(Render, per
deployment)**

- GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET
- APPLE_CLIENT_ID, APPLE_TEAM_ID, APPLE_KEY_ID, APPLE_PRIVATE_KEY
- Callback URLs registered with customer's Google/Apple org

Key files

backend/src/routes/oauth.js src/pages/Login.jsx

Phase 05	White-Label (Per Deployment)	MEDIUM
Data — DeploymentSettings singleton	• app_name, logo_url, favicon_url, primary_color, accent_color, support_email, default_locale	
Backend APIs	• GET /app/public-settings (no auth) → branding + app_name for login/landing • PUT /admin/platform/branding (system_admin only) → update settings + logo upload	
Frontend	• On boot (AuthContext or new BrandingProvider): fetch public settings → set CSS variables (--primary etc.), document.title, favicon, login logo • Extend ThemeContext or sibling BrandingContext — keep user dark/light separate from org colors • Admin UI under /admin/platform/branding: logo upload, color pickers, preview • Update siloDocxExport.js footer to use app_name from settings	
What you must provide	• App name (text), Logo (PNG/SVG transparent, ~200px wide) • Favicon (ICO or PNG 32×32), Primary/accent colors (hex) • Support email, Custom domain (CNAME to Vercel — optional)	
Key files	src/lib/BrandingContext.jsx backend/src/routes/index.js (public-settings) src/pages/admin/PlatformBranding.jsx src/lib/siloDocxExport.js	

Phase 06	Polish & Hardening	FINAL
Security & access	• Audit all routes with role matrix; wire ProtectedRoute consistently across all pages • Backend must enforce roles on /admin/* APIs — never trust UI alone • Session invalidation on disable user (optional token blocklist)	
Admin audit logs	• Log admin actions: invite, role change, disable, branding update • EmailLog for all transactional email attempts (audit + compliance)	
Completeness	• Complete appClient.entities for WorkspaceMember, SharedSession, DeploymentSettings • POST /auth/register remains disabled for public (invite-only enforced) • Document all env vars in backend/README.md and deployment runbook	
Key files	src/App.jsx src/lib/RoleRoute.jsx backend/README.md	

Suggested Implementation Order

#	Deliverable	Why it matters
1	User role + status + /auth/me + requireRole middleware	Unblocks all admin work
2	DeploymentSettings + public-settings API	White-label + login branding from day 1
3	/admin/org users list + invite API + accept-invite page	Core gov/enterprise requirement
4	Email sending for invites (emailService + templates)	Users know they are invited
5	Usage dashboard for org_admin	Monitoring per user
6	Google + Apple SSO (invite-only)	SSO without open signup
7	/admin/platform refactor (AI + branding)	System admin console

Customer Onboarding Checklist (What You Must Provide)

Mail System Setup

■ Choose delivery provider	Resend (recommended) or AWS SES or console (dev only)
■ Create Resend account / SES config	Resend.com → API key AWS: verify domain in SES
■ Verify sending domain DNS	Add SPF, DKIM, optionally DMARC records from provider
■ Choose From address	e.g. noreply@agency.gov or notifications@agency.gov
■ Set Render env vars	EMAIL_PROVIDER, RESEND_API_KEY, EMAIL_FROM, EMAIL_FROM_NAME, EMAIL_REPLY_TO

SSO (Google / Apple)

■ Google Cloud Console project	OAuth 2.0 Client ID (Web) → GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET
■ Authorized redirect URI	https://YOUR-RENDER-URL/auth/google/callback
■ Apple Developer Program (\$99/yr)	Services ID + Sign in with Apple key → APPLE_CLIENT_ID, APPLE_TEAM_ID, APPLE_KEY_ID, APPLE_PRIVATE_KEY
■ Apple redirect URI	https://YOUR-RENDER-URL/auth/apple/callback

White-Label Assets

■ App display name	Text shown in UI, emails, DOCX footer — e.g. "Ministry Meeting Intelligence"
■ Logo file	PNG/SVG, transparent background, ~200px wide
■ Favicon	ICO or PNG 32×32 for browser tab
■ Primary / accent colors	Hex codes — e.g. #1e40af

■ Support email	Shown in email footers and Settings page
-----------------	--

Infrastructure (confirm access)

■ Vercel project	VITE_API_URL pointing to Render API
■ Render web service	Ability to set env secrets
■ MongoDB connection	MONGODB_URI in Render
■ JWT secret	Strong JWT_SECRET (generate once, store securely)

People & Process

■ First system_admin email	Designate before seed script runs on production DB
■ Pilot user list	Spreadsheet: email + role (member / org_admin) for first invites
■ Customer security review	Approve OAuth apps, email domain, data residency (SES region if required)
■ Testers	2–3 email addresses for invite + SSO acceptance tests

What NOT to Build (v1)

✗ Multi-tenant Organization table with tenant_id on every row	You chose one instance per customer — not needed
✗ Stripe / consumer billing plans	Already removed from UI direction
✗ Separate admin SPA / separate repo	Unnecessary for v1 — same SPA, route groups
✗ Google Calendar / Outlook integration	Already flagged calendarIntegrations: false in featureFlags.js
✗ Self-hosted SMTP / Postfix on Render	Poor deliverability — use pluggable provider adapter
✗ Email sending from the frontend	Security and audit trail issues — backend only
✗ Marketing newsletters or digest emails	Out of scope — use mailto: for follow-up drafts

Architecture: One React SPA · Route groups + server-enforced roles · One Render backend per customer deployment · No multi-tenancy (one implicit org per instance) · Provider API keys stay in Render env secrets, never in MongoDB or UI.